



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



SIMATS ENGINEERING

MCQ Test

COURSE NAME: Software Testing

COURSE CODE: CSA3711

COURSE OUTCOME COVERED: CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Assessment Tool Weightage: 20%

Code of Conduct

I Rakcha A (192311421) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Rakcha A**

Q1. A team applies systematic test design techniques during development. This reflects:

- a) Debugging
- b) Testing as an engineering activity
- c) Maintenance
- d) Deployment

Answer: b) Testing as an engineering activity

Explanation: Systematic, technique-based test design is what makes testing an engineering activity.

Q2. If a project skips defined testing processes, what is the likely outcome?

- a) Faster delivery with high quality
- b) Increased defects in production
- c) Reduced testing cost only
- d) Improved documentation

Answer: b) Increased defects in production

Explanation: Without a defined process, defects are missed and leak into production.

Q3. Selecting test cases based on requirements demonstrates:

- a) Random testing
- b) Process-driven testing
- c) Ad hoc testing
- d) Maintenance testing

Answer: b) Process-driven testing

Explanation: Deriving tests from requirements follows a planned, process-driven approach.

Q4. Applying equivalence partitioning is an example of:

- a) Debugging
- b) Test design technique
- c) Coding standard
- d) Deployment activity

Answer: b) Test design technique

Explanation: Equivalence partitioning is a black-box test design technique.

Q5. A tester identifies missing requirements during testing. This shows:

- a) Coding error
- b) Process gap
- c) User error
- d) Hardware issue

Answer: b) Process gap

Explanation: Requirements that were never captured point to a gap in the requirement/review process.

Q6. Following a structured testing lifecycle ensures:

- a) Random defect detection
- b) Improved software quality
- c) Less documentation
- d) No defects

Answer: b) Improved software quality

Explanation: A structured lifecycle improves quality; it cannot guarantee zero defects.

Q7. Which activity best represents testing as a process?

- a) Writing code
- b) Executing test cases systematically
- c) Deploying software
- d) User training

Answer: b) Executing test cases systematically

Explanation: Systematic execution of planned test cases is the core testing process activity.

Q8. A tester reuses test cases from a repository. This supports:

- a) Efficiency in testing
- b) Increased coding errors
- c) Reduced coverage
- d) Random testing

Answer: a) Efficiency in testing

Explanation: Reuse saves design effort and time, so testing becomes more efficient.

Q9. If defects are tracked and analyzed, it improves:

- a) Coding speed
- b) Software quality process
- c) Deployment time
- d) User interface

Answer: b) Software quality process

Explanation: Defect analysis feeds process improvement and hence software quality.

Q10. Applying boundary value analysis helps to:

- a) Increase documentation
- b) Detect edge-case defects
- c) Reduce execution time
- d) Avoid testing

Answer: b) Detect edge-case defects

Explanation: BVA targets the edges of input ranges where defects commonly occur.

Q11. A tester validates system behavior against requirements. This is:

- a) Verification
- b) Validation
- c) Debugging
- d) Coding

Answer: b) Validation

Explanation: Validation checks that the built product meets the user's requirements.

Q12. Detecting defects before release reduces:

- a) Development effort
- b) Maintenance cost
- c) Coding standards
- d) Testing scope

Answer: b) Maintenance cost

Explanation: Defects found earlier mean fewer post-release fixes and lower maintenance cost.

Q13. Applying testing principles ensures:

- a) Zero defects guarantee
- b) Early defect detection
- c) No testing needed
- d) Faster coding

Answer: b) Early defect detection

Explanation: Principles such as early testing improve detection; they never guarantee zero defects.

Q14. Which principle suggests testing all inputs is impossible?

- a) Exhaustive testing
- b) Exhaustive testing is impossible
- c) Early testing
- d) Defect clustering

Answer: b) Exhaustive testing is impossible

Explanation: Testing every input and precondition is infeasible, so testing must be risk-based.

Q15. A tester focuses on frequently failing modules. This uses:

- a) Random testing
- b) Defect clustering principle
- c) Regression testing
- d) Smoke testing

Answer: b) Defect clustering principle

Explanation: Defects cluster in a few modules, so effort is concentrated there.

Q16. If testing starts early in SDLC, it helps:

- a) Reduce defects cost
- b) Increase coding errors
- c) Delay project
- d) Skip documentation

Answer: a) Reduce defects cost

Explanation: Early detection makes defects cheaper to fix.

Q17. A tester designs test cases independently from developers. This ensures:

- a) Bias
- b) Objectivity
- c) Delay
- d) Redundancy

Answer: b) Objectivity

Explanation: Independent testing gives an unbiased, objective view of the product.

Q18. A tester reviews requirements for errors. Role performed is:

- a) Developer
- b) Tester
- c) Manager
- d) User

Answer: b) Tester

Explanation: Reviewing requirements for ambiguity and errors is a tester's early-phase role.

Q19. Reporting defects clearly is part of:

- a) Coding
- b) Tester's responsibility
- c) Deployment
- d) Maintenance

Answer: b) Tester's responsibility

Explanation: Clear, reproducible defect reports are a core tester responsibility.

Q20. A tester collaborates with developers to fix bugs. This shows:

- a) Isolation
- b) Teamwork
- c) Delay
- d) Redundancy

Answer: b) Teamwork

Explanation: Collaboration between testers and developers is teamwork.

Q21. A requirement ambiguity causes defects. Origin is:

- a) Coding
- b) Requirements phase
- c) Testing phase
- d) Deployment

Answer: b) Requirements phase

Explanation: Ambiguity is introduced in the requirements phase.

Q22. Incorrect algorithm implementation leads to:

- a) Requirement defect
- b) Design defect
- c) Coding defect
- d) Testing defect

Answer: c) Coding defect

Explanation: A wrongly implemented algorithm is a mistake made in coding.

Q23. Missing validation checks are example of:

- a) Design defects
- b) Coding defects
- c) Requirement defects
- d) Deployment defects

Answer: b) Coding defects

Explanation: Validation checks that were not written in the code are coding defects.

Q24. Poor UI design causing confusion is:

- a) Coding defect
- b) Design defect
- c) Requirement defect
- d) Hardware defect

Answer: b) Design defect

Explanation: Confusing UI results from poor design decisions.

Q25. A system crash due to memory leak is:

- a) Requirement defect
- b) Coding defect
- c) Design defect
- d) Documentation defect

Answer: b) Coding defect

Explanation: Memory leaks come from faulty memory handling in the code.

Q26. Classifying defects based on severity helps:

- a) Ignore defects
- b) Prioritize fixing
- c) Delay release
- d) Increase cost

Answer: b) Prioritize fixing

Explanation: Severity tells which defects must be fixed first.

Q27. A critical defect impacts:

- a) Minor feature
- b) Core functionality
- c) UI color
- d) Documentation

Answer: b) Core functionality

Explanation: Critical defects break core functionality.

Q28. A minor defect example is:

- a) System crash
- b) Login failure
- c) Typo error
- d) Data loss

Answer: c) Typo error

Explanation: A typo does not affect functionality, so it is minor.

Q29. Defect classification helps testers to:

- a) Ignore bugs
- b) Organize defects
- c) Skip testing
- d) Increase errors

Answer: b) Organize defects

Explanation: Classification organizes defects for analysis and handling.

Q30. A defect repository is used to:

- a) Store code
- b) Store test cases
- c) Store defect details
- d) Store UI designs

Answer: c) Store defect details

Explanation: A defect repository stores details of reported defects.

Q31. Reusing defect data helps in:

- a) Predicting issues
- b) Increasing bugs
- c) Reducing tests
- d) Skipping phases

Answer: a) Predicting issues

Explanation: Past defect data helps predict where problems will occur.

Q32. A defect repository improves:

- a) Code complexity
- b) Testing efficiency
- c) User interface
- d) Deployment speed

Answer: b) Testing efficiency

Explanation: Historical defects guide better, more efficient testing.

Q33. Logging defects with details ensures:

- a) Confusion
- b) Traceability
- c) Delay
- d) Redundancy

Answer: b) Traceability

Explanation: Detailed logs let each defect be traced through its life cycle.

Q34. A tester uses past defect data to design tests. This is:

- a) Random testing
- b) Experience-based testing
- c) Debugging
- d) Deployment

Answer: b) Experience-based testing

Explanation: Using past experience and defect history is experience-based testing.

Q35. Incorrect login validation is an example of:

- a) Functional defect
- b) Performance defect
- c) Security defect
- d) UI defect

Answer: a) Functional defect

Explanation: The login function does not behave as specified, so it is a functional defect.

Q36. Slow response time is a:

- a) Functional defect
- b) Performance defect
- c) UI defect
- d) Coding standard

Answer: b) Performance defect

Explanation: Response time is a performance attribute.

Q37. Unauthorized data access indicates:

- a) UI defect
- b) Security defect
- c) Functional defect
- d) Design defect

Answer: b) Security defect

Explanation: Access control failure is a security defect.

Q38. Missing button functionality is:

- a) Functional defect
- b) Security defect
- c) Performance defect
- d) Hardware defect

Answer: a) Functional defect

Explanation: The feature does not work as specified, so it is functional.

Q39. Poor alignment in UI is:

- a) Functional defect
- b) UI defect
- c) Security defect
- d) Design defect

Answer: b) UI defect

Explanation: Alignment is a visual/layout (UI) issue.

Q40. Applying testing principles avoids:

- a) Coding
- b) Redundant tests
- c) Deployment
- d) Maintenance

Answer: b) Redundant tests

Explanation: Principles guide focused testing and avoid wasteful, redundant tests.

Q41. A tester validates outputs using expected results. This is:

- a) Debugging
- b) Test execution
- c) Coding
- d) Maintenance

Answer: b) Test execution

Explanation: Comparing actual with expected results is test execution.

Q42. If defects are not tracked, it leads to:

- a) Better quality
- b) Poor traceability
- c) Faster delivery
- d) No impact

Answer: b) Poor traceability

Explanation: Untracked defects cannot be followed, analysed or traced.

Q43. Applying test process ensures:

- a) Random results
- b) Structured testing
- c) No defects
- d) No planning

Answer: b) Structured testing

Explanation: A test process gives structure to testing.

Q44. Test case design based on requirements ensures:

- a) Low coverage
- b) High coverage
- c) No defects
- d) Less effort

Answer: b) High coverage

Explanation: Requirement-based design gives good requirement coverage.

Q45. Early testing reduces:

- a) Defect cost
- b) Code size
- c) UI design
- d) Deployment time

Answer: a) Defect cost

Explanation: Early detection lowers the cost of defects.

Q46. A tester identifies repeated defects. This indicates:

- a) Good design
- b) Poor process
- c) No issue
- d) Perfect testing

Answer: b) Poor process

Explanation: Recurring defects show weaknesses in the process.

Q47. Reviewing test results helps:

- a) Ignore defects
- b) Improve quality
- c) Skip testing
- d) Reduce effort

Answer: b) Improve quality

Explanation: Analysing results leads to corrective actions and better quality.

Q48. A tester applies regression testing to:

- a) Add features
- b) Ensure fixes work
- c) Deploy software
- d) Design UI

Answer: b) Ensure fixes work

Explanation: Regression testing confirms that fixes/changes did not break existing behaviour.

Q49. Identifying root cause of defects helps:

- a) Increase bugs
- b) Prevent recurrence
- c) Delay testing
- d) Skip coding

Answer: b) Prevent recurrence

Explanation: Root-cause analysis lets the team prevent the same defect again.

Q50. A test plan ensures:

- a) Random execution
- b) Organized testing
- c) No testing
- d) No defects

Answer: b) Organized testing

Explanation: A test plan organizes scope, approach, resources and schedule.

Q51. Applying defect classification improves:

- a) Testing cost
- b) Fix prioritization
- c) Coding errors
- d) UI design

Answer: b) Fix prioritization

Explanation: Classification supports prioritizing fixes.

Q52. A tester uses historical defect trends. This helps in:

- a) Ignoring defects
- b) Risk-based testing
- c) Deployment
- d) Coding

Answer: b) Risk-based testing

Explanation: Trends show high-risk areas to test first.

Q53. Testing ensures product meets:

- a) Developer needs
- b) User requirements
- c) Tester goals
- d) Manager plans

Answer: b) User requirements

Explanation: Testing checks that the product satisfies user requirements.

Q54. A defect due to wrong requirement interpretation is:

- a) Coding defect
- b) Requirement defect
- c) UI defect
- d) Hardware defect

Answer: b) Requirement defect

Explanation: The origin is misunderstanding/ambiguity in the requirements.

Q55. Applying process standards ensures:

- a) Poor quality
- b) Consistent quality
- c) No testing
- d) Random output

Answer: b) Consistent quality

Explanation: Standards bring consistency to quality.

Q56. A tester documents defects clearly. This helps:

- a) Developers fix quickly
- b) Increase errors
- c) Delay release
- d) Reduce quality

Answer: a) Developers fix quickly

Explanation: Clear documentation lets developers reproduce and fix faster.

Q57. A defect repository is analyzed to:

- a) Increase cost
- b) Improve testing strategy
- c) Skip testing
- d) Reduce quality

Answer: b) Improve testing strategy

Explanation: Analysis of past defects improves the test strategy.

Q58. A tester applies black-box testing. This focuses on:

- a) Internal code
- b) External behavior
- c) Hardware
- d) Deployment

Answer: b) External behavior

Explanation: Black-box testing checks external behaviour without looking at the code.

Q59. Defects found in testing phase indicate:

- a) Perfect design
- b) Earlier phase issues
- c) No issues
- d) Hardware problems

Answer: b) Earlier phase issues

Explanation: Defects found in testing usually originated in earlier phases.

Q60. Applying structured testing improves:

- a) Random results
- b) Product reliability
- c) Coding errors
- d) Deployment delay

Answer: b) Product reliability

Explanation: Structured testing improves reliability.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand